Calibrated Perplexity for Robust AI-Generated Text Detection Across Languages and Genres

Vladimir Chaikin
MIPT
Moscow, Russia
email@phystech.edu
& *AnastasiaVoznyuk*
MIPT
Moscow, Russia
email@phystech.edu

May 6, 2026

Abstract

This paper invistigates the problem of robust AI-generated text detection under distribution shift. While simple perplexity is a simple solution for such detection, its oppurtunities lose significantly on out-of-domain data. We propose a novel method - calibrated perplexity - which normalizes raw perplexity scores using domain-specific language model baselines. This approach constructs a stable feature space less sensitive to text length, genre, and language variations. Experiments across multiple domains and languages are going to demonstrate improved in-domain and out-of-domain detection accuracy compared to standard perplexity-based methods.

Keywords: Calibrated perplexity, Cross-domain detection, Text classification, Genre adaptation

1 Introduction

Fast improvement and widespread accessibility of Large Language Models (LLMs) have destroyed the lines between human-written and machine-generated text. While this technology offers benefits, it also presents significant disadvantages, including academic dishonesty, disinformation, and the fall of trust in digital content [5, 4]. This has created an inevitable need for robust methods to detect AI-generated content.

Current detectors often rely on fine-tuned classifiers or statistical signatures taken from the text [2]. Useful and computationally efficient feature is perplexity, a measure of how "surprising" a text is to a language model. The assumption is

that LLMs, which are trained to maximize likelihood, will generate text with lower perplexity compared to human writing. As it is shown in [1], a significant change comes from distribution shift. The perplexity of a text is not an absolute value; it is highly dependent on the domain (e.g., news, scientific articles, social media) and language it was trained on. A model fine-tuned for one genre often fails on another, causing unreliable detection in real world, where the target domain is unknown.

Recent works have attempted to solve this by developing zero-shot detectors that are more robust to unseen prompts or models. For instance, Binoculars [4] uses a ratio of scores from two different LLMs for calibration. Similarly, research on multilingual and cross-domain detection highlights the necessity of adapting methods to diverse data distributions [3]. Despite these advances, a method for calibrating perplexity for each domain itself to make it robust across different genres and languages remains still not-explored.

In this paper, we face the challenge of domain shift and try to avoid its effect on detection. We propose a novel method called calibrated perplexity, which normalizes perplexity scores against domain-specific baselines. The final goal is not to find out one universal calibrated perplexity score, but to get a set of parameters to have an opportunity to learn fastly new zero-shot models. Our approach includeses training KenLM n-gram models on text from multiple domains and using these models to compute a relative, rather than absolute, measure of text predictability. The core novelty of our work lies in transforming a domain-dependent score into a stable feature for classification. To check our method, we conduct experiments across multiple domains and languages, comparing in-domain and out-of-domain detection.

2 Problem statement

Let $\mathbf{W}$ be our alphabet, tokens consist of characters from that alphabet.

Let

$$\mathbb{D} = \left\{ [t_j]_{j=1}^{n} \;\middle|\; t_j \in \mathbf{W},\ n \in \mathbb{N} \right\}$$

be the space of our documents.

We have a set of N documents

$$\mathbf{D} = \bigcup_{i=1}^{N} D^i, \quad D^i \in \mathbb{D}.$$

We assume that each document may either be fully written by a human or generated by an AI model. Accordingly, we introduce a binary classification setup, where the label 0 denotes a human-written fragment and 1 denotes a machine-generated fragment. Let $\mathbf{C} = \{0, 1\}$ be the set of these labels.

To capture the statistical properties of text fragments, we consider a probabilistic model of the language. For a given fragment $F = [t_a, t_{a+1}, \ldots, t_b]$ we define its probability as the product of conditional probabilities of each token

given its $n-1$ predecessors:

$$P(F) = \prod_{i=a}^{b} P(t_i \mid t_{i-1}, \ldots, t_{i-n+1}).$$

The choice of such an n-gram model is motivated in [3], which states that generative language models tend to sample from the head of the token distribution, i.e., they prefer high-probability continuations. This makes n-gram statistics a natural baseline for distinguishing human-written from machine-generated text.

The log-probability of a fragment is then

$$\log P(F) = \sum_{i=a}^{b} \log P(t_i \mid t_{i-1}, \ldots, t_{i-n+1}),$$

and the perplexity is defined as the inverse geometric mean of the token probabilities:

$$\mathrm{PPL}(F) = P(F)^{-\frac{1}{|F|}} = 2^{-\frac{1}{|F|}\log_2 P(F)},$$

where $|F| = b - a + 1$ is the length of the fragment in tokens.

Perplexity quantifies how "surprising" a fragment is under the language model: lower values indicate that the fragment is more predictable.

Given a training set of documents $\mathbf{D}_{\text{train}} \subset \mathbb{D}$ with known labels, we learn the conditional probabilities $P(t_i \mid t_{i-1}, \ldots, t_{i-n+1})$ by maximizing the likelihood of the training data.

The training objective is the negative log-likelihood of the observed documents, which is equivalent to minimizing the cross-entropy between the model distribution and the empirical distribution of the training data. This optimization is typically performed using count-based estimation with appropriate smoothing to handle sparse n-gram occurrences.

For classification, we introduce a threshold τ_d for each domain d. A fragment F is classified as machine-generated if:

$$\mathrm{PPL}_{\mathcal{M}_d}(F) < \tau_d,$$

and as human-written otherwise. The threshold τ_d is selected on a validation set to maximize the F1-score, balancing precision and recall. $\mathcal{M}_d$ is n-gram model providing probabilities of tokens

3 Method

3.1 Overview

We propose an ensemble-based method for AI-generated text detection that operates without knowledge of the test text's domain. The method consists of three stages:

1. **Training:** Train separate n-gram language models on different domains using only human-written texts.
2. **Threshold calibration:** For each domain model, select a classification threshold that maximizes F1-score on a validation set from the same domain.
3. **Ensemble inference:** For a new text, compute perplexities from all domain models and combine their predictions using one of three ensemble strategies.

3.2 Notation

Let $\mathcal{D} = \{d_1, d_2, \ldots, d_M\}$ be a set of M domains (e.g., news, essays, scientific texts). For each domain d, we have:

- Training set $\mathbf{T}_d \subset \mathbb{D}$ consisting of human-written texts from domain d
- Validation set $\mathbf{V}_d \subset \mathbb{D}$ with balanced human and machine texts from domain d
- Test set $\mathbf{X}_d \subset \mathbb{D}$ held-out from the same distribution as validation

For a text x, let $\mathrm{PPL}_{\mathcal{M}_d}(x)$ denote the perplexity computed by the language model trained on domain d.

3.3 Model Training

For each domain $d \in \mathcal{D}$, we train a trigram language model $\mathcal{M}_d$ using KenLM with modified Kneser-Ney smoothing. The training data consists exclusively of human-written texts from that domain:

$$\mathcal{M}_d = \mathrm{KenLM}(\mathbf{T}_d), \quad \text{where } \mathbf{T}_d = \{x \in \mathbb{D} \mid \mathrm{source}(x) = \mathrm{human}, \mathrm{domain}(x) = d\}.$$

The training objective is to minimize the negative log-likelihood of the training corpus:

$$\mathcal{L}(\mathcal{M}_d) = -\sum_{x \in \mathbf{T}_d} \log P_{\mathcal{M}_d}(x) \to \min.$$

3.4 Threshold Calibration

For each model $\mathcal{M}_d$, we find an optimal threshold τ_d that maximizes the F1-score on the validation set $\mathbf{V}_d$:

$$\tau_d = \arg\max_{\tau} \mathrm{F1}\left(\mathbf{1}[\mathrm{PPL}_{\mathcal{M}_d}(x) < \tau],\ y(x)\right),$$

where $y(x) = 1$ for machine-generated texts and $y(x) = 0$ for human-written texts.

The F1-score is defined as the harmonic mean of precision and recall:

$$\mathrm{F1} = 2 \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}.$$

In practice, we search over 200 evenly spaced thresholds between the minimum and maximum perplexity values observed on the validation set.

3.5 Ensemble Inference

For a test text x with unknown domain, we compute perplexities $p_d = \mathrm{PPL}_{\mathcal{M}_d}(x)$ for all $d \in \mathcal{D}$. These perplexities are then used in three complementary ensemble strategies.

3.5.1 Strategy A: Simple Majority Voting

This is the baseline ensemble. Each model makes an individual prediction $h_d(x) = \mathbf{1}[p_d < \tau_d]$, where 1 indicates "machine-generated". The final decision is based on simple majority voting:

$$H_{\mathrm{majority}}(x) = \mathbf{1}\left[\sum_{d \in \mathcal{D}} h_d(x) \geq \lceil M/2 \rceil\right],$$

where $M = |\mathcal{D}|$ is the number of domains. A text is classified as machine-generated if at least half of the models vote for machine.

3.5.2 Strategy B: Native Domain Detection

This strategy is based on the observation that a text originating from a particular domain yields minimal perplexity for the model trained on that domain. We first identify the *native domain* of the text:

$$d^* = \arg\min_{d \in \mathcal{D}} p_d.$$

Then we use only the prediction of the corresponding model:

$$H_{\mathrm{native}}(x) = \mathbf{1}[p_{d^*} < \tau_{d^*}].$$

This approach effectively performs domain detection before classification, using the model with the lowest perplexity as the most reliable for the given text.

3.5.3 Strategy C: Weighted Voting by Domain Significance

We assign weights to each model based on its ability to distinguish between human and machine texts. The weight w_d for domain d is proportional to the

difference between the model's true positive rate and false positive rate on the validation set:

$$w_d = \frac{\mathrm{TPR}_d - \mathrm{FPR}_d}{\sum_{d' \in \mathcal{D}} (\mathrm{TPR}_{d'} - \mathrm{FPR}_{d'})},$$

where TPR_d is the true positive rate (recall on machine texts) and FPR_d is the false positive rate (errors on human texts) for model $\mathcal{M}_d$. Models that better distinguish between classes receive higher weights.

The weighted prediction is:

$$H_{\text{weighted}}(x) = \mathbf{1}\left[\sum_{d \in \mathcal{D}} w_d \cdot \mathbf{1}[p_d < \tau_d] \geq 0.5\right].$$

3.6 Properties

The proposed method has the following properties:

- **Domain-agnostic inference:** No prior knowledge of the test text's domain is required.
- **Interpretability:** Each model provides a clear per-domain prediction; low perplexity indicates that the text "belongs" to that domain.
- **Robustness:** Weighted voting reduces the impact of models that perform poorly on the input text.
- **Scalability:** Adding a new domain requires training one additional KenLM model and computing its validation statistics.

3.7 Optimization Problem

We aim to find model parameters and thresholds that minimize the variance of predictions across different domain models for the same text:

$$\mathcal{L} = \sum_{i<j} \left(\mathbf{1}[p_{d_i} < \tau_{d_i}] - \mathbf{1}[p_{d_j} < \tau_{d_j}]\right)^2 \to \min.$$

In practice, we approximate this ideal by constructing ensemble strategies that balance the contributions of individual models. When all models agree on the classification, the confidence is high; when they disagree, the ensemble combines their predictions in a weighted manner to achieve a robust decision.

4 Experiments

4.1 Experimental Setup

We conduct experiments on a collection of Russian-language texts spanning seven domains: news, scientific texts, stories, articles, hard_news, hard_science,

hard_essays. The dataset is balanced with approximately 3,500 human-written and 3,500 machine-generated texts per domain. Texts shorter than 50 words are filtered out. All models are trained using KenLM with trigram order ($n = 3$) and modified Kneser-Ney smoothing.

4.2 Experiment 1: Single-Domain Training and In-Domain Testing

For each domain, we train a KenLM model exclusively on human-written texts from that domain. We then evaluate the model on a held-out test set from the same domain, using a threshold optimized by F1-score on a validation set.

The optimal F1-thresholds for each domain are as follows:

Table 1: Optimal thresholds and F1-scores for each domain (in-domain evaluation).

Domain	**Optimal threshold τ_d**	**F1-score**
AINL science	8.28	0.85
Stories	12.09	0.78
Articles	9.60	0.73
News	8.61	0.73
Hard science	11.56	0.83
Hard essays	8.91	0.72
Hard news	12.10	0.75

These thresholds are used in all subsequent ensemble experiments. The substantial variation in optimal threshold values (from 8.28 to 12.10) confirms that perplexity is highly domain-dependent.

4.3 Experiment 2: Cross-Domain Testing

We evaluate each model on test sets from all other domains, using thresholds calibrated on the source domain.

Results: Cross-domain performance drops substantially. For example, the model trained on news achieves an F1-score of only 0.69 when tested on scientific texts, compared to 0.85 in-domain. This confirms that perplexity thresholds are domain-dependent and cannot be directly transferred.

4.4 Experiment 3: Generalization on Mixed-Domain Data

We combine all domains into a single balanced dataset (7,056 texts total, split 70/15/15 for train/validation/test). A single KenLM model is trained on the combined training set.

Results: The unified model achieves a balanced accuracy of 0.68 and F1-score of 0.68. While this outperforms cross-domain transfer, it still lags behind in-

domain performance on individual domains, indicating that domain heterogeneity remains a challenge.

4.5 Experiment 4: Tokenization and Pruning Optimization

We experiment with SentencePiece tokenization (vocab_size=8000) and n-gram pruning ($-prune$ 2 2 2) to improve model stability across domains.

Results: Combined optimization yields a modest improvement of 1-2% in balanced accuracy (from 0.68 to 0.69), suggesting that while helpful, tokenization and pruning alone are insufficient to fully resolve domain shift.

4.6 Experiment 5: Ensemble Methods

We implement three ensemble strategies combining the seven domain-specific models:

Table 2: Comparison of ensemble methods on the mixed-domain test set.

Method	**Acc**	**F1**	**Human recall**	**AI recall**
Simple majority voting (Strategy A)	0.76	0.75	79.1%	72.2%
Native domain detection (Strategy B)	0.76	0.75	82.7%	69.0%
Weighted voting (Strategy C)	0.76	0.75	79.2%	71.8%

Observations:

- **Strategy A (Simple majority voting)**: A text is classified as machine-generated if at least one model votes for machine. This achieves 79.1% human recall and 72.2% AI recall.
- **Strategy B (Native domain detection)**: For each text, we identify the model with minimal perplexity (the "native" domain) and use its individual prediction. This yields the highest human recall (82.7%) with AI recall of 69.0%.
- **Strategy C (Weighted voting)**: Weights are assigned based on each model's discriminative power (TPR - FPR) on the validation set. This method achieves balanced performance with 79.2% human recall and 71.8% AI recall.

All three ensemble strategies achieve comparable overall accuracy (0.76) and F1-score (0.75), demonstrating the robustness of the ensemble approach. The trade-off lies in class-specific performance: native domain detection favors human texts, while simple majority voting is more balanced.

4.7 Summary of Key Findings

1. **Domain shift is significant:** Optimal thresholds vary substantially across domains, ranging from 8.28 (AINL science) to 12.10 (hard news).

2. **Single-domain models do not generalize:** Cross-domain performance drops significantly compared to in-domain evaluation.

3. **Ensemble methods balance the trade-off:** All three ensemble strategies achieve 0.76 accuracy and 0.75 F1-score, with different class-specific biases.

4. **Native domain detection excels at human recall:** Identifying the domain with minimal perplexity and using its prediction achieves 82.7% human recall.

5. **Simple majority voting provides balanced performance:** With 79.1% human recall and 72.2% AI recall, this is the simplest and most robust approach.

5 Discussion

Our experiments show that perplexity-based detection suffers from significant domain shift: optimal thresholds vary by nearly a factor of 1.5 across domains (8.28 to 12.10), making a single universal threshold not practical.

The three ensemble strategies each address this problem differently:

- **Simple majority voting** achieves balanced performance (79% human recall, 72% AI recall) with 0.76 accuracy.
- **Native domain detection** (using the model with minimal perplexity) yields the highest human recall (83%) at 69% AI recall.
- **Weighted voting** provides similar balanced results.

All ensembles outperform single-domain models applied out-of-domain, confirming that combining domain-specific models effectively smoothes domain shift.

Limitations: Our work is limited to Russian language, seven domains, and KenLM as the base model. Results may not directly generalize to other languages or neural models.

Future work: Comparing with other models, adaptive threshold selection for unseen domains.

6 Conclusion

We addressed domain shift in perplexity-based AI-generated text detection. Our main findings are:

1. Optimal perplexity thresholds vary significantly across domains (8.28–12.10), confirming domain dependence.

2. Three ensemble strategies combining domain-specific KenLM models achieve robust detection without knowledge of the test text's domain.

3. Simple majority voting achieves 79% human recall and 72% AI recall (accuracy 0.76, F1 0.75).

4. Native domain detection achieves the highest human recall (83%).

All ensemble methods outperform single-domain models on out-of-domain data, providing a practical solution for AI-generated text detection.

References

[1] O. Bakhteev et al. Automatic detection of machine generated texts: Need more tokens. *2022 Ivannikov Memorial Workshop (IVMEM)*, 2022.

[2] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. Gltr: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.

[3] G. M. Gritsay, A. V. Grabovoy, A. S. Kildyakov, and Yu. V. Chekhovich. Artificially generated text fragments search in academic documents. *Doklady Mathematics*, 108:S434–S442, 2023.

[4] Abhimanyu Hans et al. Spotting llms with binoculars: Zero-shot detection of machine-generated text. *arXiv preprint arXiv:2401.12070*, 2024.

[5] Z. Xu and V. S. Sheng. Detecting ai-generated code assignments using perplexity of large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21):23155–23162, 2024.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems

TODO